EMPLOYEE PERFORMANCE EVALUATION SYSTEM

Case Study

Presented by Ayush Thakare & Sarang Wasamwar
[124B1B094] [124B1B090]

Introduction

- The Employee Performance Evaluation System is designed to manage:
  - Employee details
  - Department assignments
  - Performance reviews
  - Skill-Based Evaluations
  - Training and Certifications
  - Rewards and Recognition
- The project contains multiple interrelated operations.
- Many of these operations involve dependent database actions.
- Such operations must be executed as transactions to maintain correctness.
- This section focuses on:
  - Identifying transactions in the project
  - Analyzing transaction schedules
  - Checking serializability
  - Ensuring ACID properties
  - Understanding concurrency, rollback, recovery, and deadlock concepts

Identification of Real Transactions in Our Project

Major Transactions Present in Our Project:

1. Add New Employee
Insert employee record
Assign department
Assign manager (if applicable)

2. Assign Employee to Project
Verify employee exists
Verify project exists
Insert into EmployeeProject

3. **Submit Employee Review**
Insert into Review
Insert multiple rows into ReviewSkillScore
Optionally add/update goal
Optionally add feedback

4. Enroll Employee in Training
Verify employee exists
Verify training exists
Insert into TrainingEnrollment

5. Complete Training and Issue Certification
Update TrainingEnrollment
Insert into Certification

6. Process Rewards / Compensation
Insert into Bonus
Insert into Promotion
Insert into Recognition

Key Transactions Selected for Analysis

1. Submit Employee Review (Primary Transaction):

Most important and central workflow
Involves multiple dependent operations
Best for ACID, rollback, and schedule analysis

2. Complete Training + Issue Certification:

Demonstrates dependent update + insert operations
Good example of consistency and commit/rollback

3. Process Rewards / Compensation:

Demonstrates reward-related multi-step updates
Useful for consistency and concurrent access discussion

Main Transaction Workflow

Business Scenario:

When a manager submits a performance review for an employee, the system performs multiple dependent operations:

Steps involved:

- Read employee details
- Insert a new row into Review
- Insert multiple rows into ReviewSkillScore
- Optionally add/update Goal
- Optionally add Feedback
- Commit transaction

Why this is treated as one transaction?

All review-related operations together represent one logical unit of work

If any step fails, the database must not keep partial review data

Therefore, all steps must either:

Complete Successfully, or Rollback Completely

SQL Transaction Example: Performance Review Submission

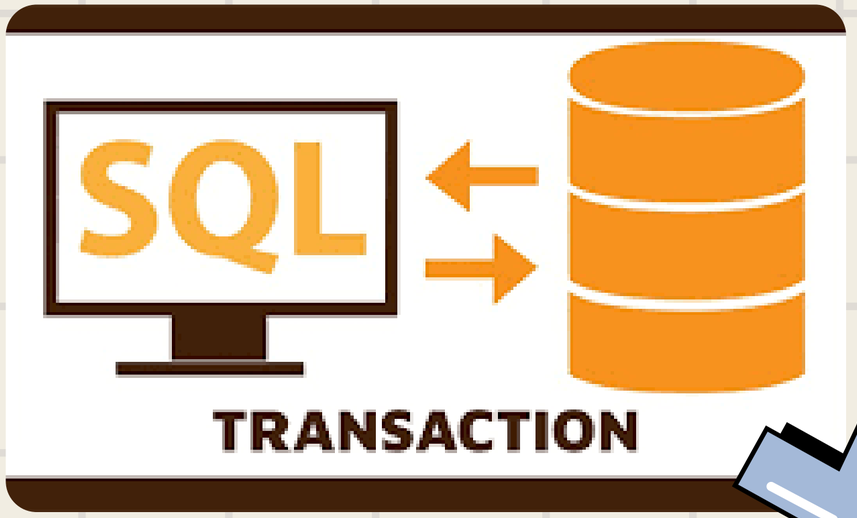

```
START TRANSACTION;

INSERT INTO Review (ReviewID, EmployeeID, ManagerID, ReviewDate, Rating)
VALUES (507, 103, 201, '2026-03-15', 4.40);

INSERT INTO ReviewSkillScore (ReviewID, SkillID, Score)
VALUES (507, 401, 4.5);

INSERT INTO ReviewSkillScore (ReviewID, SkillID, Score)
VALUES (507, 402, 4.7);

COMMIT;
```



Formal Transactions Used for Schedule Analysis

Transaction T1: Submit Review for Employee 103
T1:
1. R1(Employee 103)
2. W1(Review 507 for Employee 103)
3. W1(ReviewSkillScore for Skill 401 = 4.5)
4. W1(ReviewSkillScore for Skill 402 = 4.7)
5. C1

Transaction T2: Complete Training + Issue Certification for Employee 103
T2:
1. R2(TrainingEnrollment of Employee 103)
2. W2(Update TrainingEnrollment status = 'Completed')
3. W2(Insert Certification for Employee 103, Skill 402)
4. C2

Conflict Serializability Analysis

Definition:
A schedule is conflict serializable if it can be transformed into a serial schedule by swapping only non-conflicting operations.

Conflict Check in Our Schedule
In schedule S1:

T1 mainly accesses:
Review
ReviewSkillScore

T2 mainly accesses:
TrainingEnrollment
Certification

Observation:
T1 and T2 mostly operate on different data items
There is no harmful Read-Write / Write-Write conflict on the same item
Hence, no cyclic dependency is created between T1 and T2

Conclusion:
Schedule S1 is Conflict Serializable
Equivalent serial order can be considered as:
T1 → T2

View Serializability Analysis

Definition:
A schedule is view serializable if:
transactions read the same initial values,
final writes are preserved,
and the schedule produces the same result as some serial schedule.

Analysis of S1
In our schedule:
T1 and T2 do not read uncommitted data from each other
Final writes remain preserved
The overall result is equivalent to serial execution

Important Observation:
Since S1 is conflict serializable, it is automatically view serializable.

Conclusion:
Schedule S1 is View Serializable

ACID Properties Applied to Our Project

1. Atomicity

Review submission includes:

- insert into Review
- insert multiple rows into ReviewSkillScore
- All steps must succeed together
- If any step fails → rollback entire transaction

2. Consistency

- Review.EmployeeID must exist in Employee
- Review.ManagerID must exist in Manager
- ReviewSkillScore.ReviewID must reference valid Review
- SkillID must reference valid Skill
- Database moves from one valid state to another valid state

3. Isolation

- Concurrent transactions should not see half-completed updates
- Example:
- another manager should not see partially inserted review data
- Prevents dirty read / inconsistent intermediate state

4. Durability

- Once committed, review data remains permanently stored
- Even if system crashes after commit, the data should not be lost

SQL Transaction Example: Rollback Handling

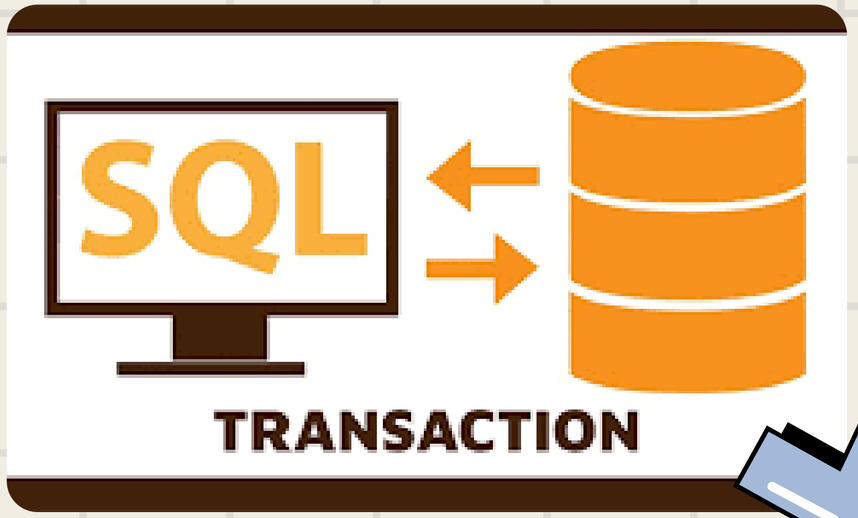

```
START TRANSACTION;

INSERT INTO Review (ReviewID, EmployeeID, ManagerID, ReviewDate, Rating)
VALUES (508, 104, 201, '2026-03-16', 4.20);

INSERT INTO ReviewSkillScore (ReviewID, SkillID, Score)
VALUES (508, 401, 4.1);

-- Suppose this insert fails due to invalid SkillID
INSERT INTO ReviewSkillScore (ReviewID, SkillID, Score)
VALUES (508, 999, 4.3);

ROLLBACK;
```



Commit, Rollback, Recovery, Deadlock & Concurrency Control

1. Commit Example
If all review-related operations succeed:
- Insert Review
- Insert ReviewSkillScore (Skill 401)
- Insert ReviewSkillScore (Skill 402)
- COMMIT
- All changes become permanent

2. Rollback Example
If second skill score insertion fails:
- Insert Review
- Insert ReviewSkillScore (Skill 401)
- Insert ReviewSkillScore (Skill 402) → FAIL
- ROLLBACK
- Previously inserted review data is undone
- No partial review remains in database

3. Recovery Example
If system crashes before commit:
- Uncommitted changes must be undone during recovery
- Database returns to consistent state

4. Deadlock Example
Possible case:
- T1 locks review-related resource for Employee 103
- T2 locks training-related resource for Employee 103
- T1 requests T2's resource
- T2 requests T1's resource
- Both wait → Deadlock

5. Concurrency Control
To avoid unsafe concurrent execution:
- Use lock-based concurrency control
- Recommended method: Two-Phase Locking (2PL)
- Other possible handling:

1. lock ordering
2. timeout
3. deadlock detection

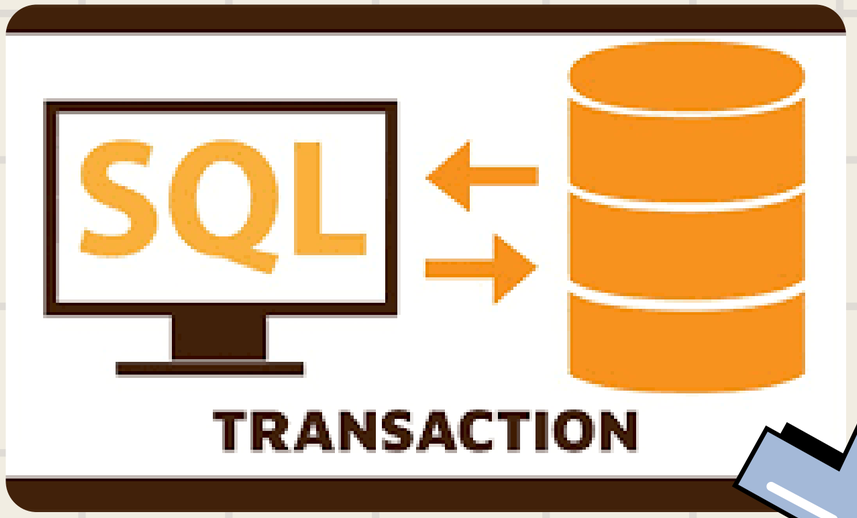
SQL Transaction Example: Update


```
START TRANSACTION;

UPDATE TrainingEnrollment
SET Status = 'Completed'
WHERE EmployeeID = 103 AND TrainingID = 802;

INSERT INTO Certification (CertificationID, EmployeeID, SkillID, CertificationName, IssueDate)
VALUES (906, 103, 402, 'SQL Specialist', '2026-03-20');

COMMIT;
```



In conclusion, the Employee Performance Evaluation System was successfully extended from a relational database model to include both transaction management and NoSQL implementation. We identified the key transactions in the project, analyzed transaction schedules, verified conflict and view serializability, and demonstrated ACID properties along with commit, rollback, recovery, and deadlock concepts. Further, the same project was redesigned in MongoDB using equivalent collections and document-based structures, where CRUD operations, MongoDB queries, aggregation, indexing, and search features were explored. Overall, the project demonstrates a clear understanding of both SQL and NoSQL database concepts while maintaining continuity with the same problem statement.

THANK YOU